Do Subscribe and Support 💕

******** Cloud Engineer Interview Questions *******

Important Note:

You will find AWS services naming convention here. If you are preparing for Azure or GCP, you can still use these questions—just change the names to match your cloud provider.

All the very best – Keep Hustling and don't lose hope

LEVEL 2 – Moderate / Slightly Difficult Questions

(Concept + Understanding + Internal Working)

1. How does traffic flow from browser to EC2?

When you type a URL in your browser, the first thing that happens is DNS resolution. Your browser asks a DNS server "what's the IP address for www.example.com?" The DNS server responds with an IP address, let's say 54.123.45.67. If you're using Route 53, it might return the IP of a load balancer.
The browser then sends an HTTP request to that IP address. This request goes out to the internet and reaches AWS's edge location. If you're using CloudFront, it might serve cached content from there. Otherwise, the request continues to the AWS region where your resources are.

The request hits the Internet Gateway attached to your VPC. The Internet Gateway receives it and checks the destination IP. If it's a public IP associated with an EC2 instance, the gateway translates that public IP to the instance's private IP using NAT.

The traffic then enters your VPC and the route table determines where it goes next. If you have a load balancer, the request goes there first. The load balancer sits in public subnets across multiple availability zones. It receives the request and decides which backend EC2 instance should handle it based on its algorithm and health checks.

The load balancer forwards the request to an EC2 instance, probably in a private subnet. Before the traffic reaches the instance, it passes through the Network ACL for that subnet - the NACL checks its rules and allows or denies based on the source IP and port. Then it reaches the instance's Security Group, which does another check. If both allow it, the traffic finally reaches your EC2 instance.

Your application on the EC2 instance processes the request and sends a response back. The response goes back through the Security Group (automatically allowed because it's stateful), through the NACL (needs explicit outbound rule because it's stateless), to the load

balancer, then back through the Internet Gateway which translates the private IP back to public, and finally back across the internet to the user's browser.

---

2. Why can't a private subnet access the internet directly?

A private subnet can't access the internet directly because its route table doesn't have a route pointing to an Internet Gateway. The route table is like a GPS for network traffic - it tells packets where to go. In a private subnet, the route table only has routes for local VPC traffic, nothing pointing to the internet.

Even if you manually tried to send traffic to an internet IP address from an instance in a private subnet, the route table would look at the destination and say "I don't have a route for this." The traffic would be dropped because there's literally no path defined for it to follow.

This is completely intentional for security. By not having a route to the Internet Gateway, you're ensuring that no inbound traffic from the internet can reach these instances. There's no routing path that would allow it. Even if someone knows your instance's private IP address, they can't connect to it from the internet because the routing infrastructure doesn't support it.

When instances in private subnets need outbound internet access - like downloading software updates or calling external APIs - they use a NAT Gateway. The route table in a private subnet has a route that says "for internet traffic (0.0.0.0/0), send it to the NAT Gateway." The NAT Gateway sits in a public subnet and has access to the Internet Gateway. It forwards the request but doesn't allow any inbound connections initiated from the internet.

Think of it like a house with no front door. You can leave through the back door to go somewhere (through NAT), but strangers can't walk in from the street because there's no door for them to enter through. The architecture physically prevents the connection at the routing level.

---

3. Difference between Security Group and NACL in real-world use?

In real-world use, Security Groups are your primary security control and what you'll configure most often. NACLs are more of a backup layer that you use for specific scenarios.

Security Groups are where you do most of your work. When you launch an EC2 instance, you attach security groups to control what can talk to it. You might have a "web-server" security group that allows HTTP and HTTPS from anywhere, and SSH from only your office. You'd have an "app-server" security group that only allows traffic from the web-server security group. And a "database" security group that only allows traffic from the app-server

security group. This is intuitive and easy to manage because you're thinking about what each instance needs.

NACLs are subnet-level and you typically use them for broader controls. A common use case is blocking malicious IP addresses. If you're getting attacked from a specific IP range, you can add a deny rule in the NACL to block it at the subnet level before it even reaches your instances. You can't do this with Security Groups because they only have allow rules.

Another real-world use for NACLs is enforcing network segmentation policies. You might have a compliance requirement that your database subnet can never communicate with the internet, even accidentally. You'd use NACL rules to enforce this at the subnet boundary as an extra safety layer, even though your Security Groups should already prevent it.

The stateless nature of NACLs can be tricky. You need to remember to allow ephemeral ports (1024-65535) for return traffic. I've seen cases where someone added an allow rule for inbound HTTP but forgot the outbound rule for responses, and nothing worked. With Security Groups, you don't have this problem because they're stateful.

In practice, most engineers set up NACLs once during initial VPC setup with basic allow rules and then rarely touch them. Security Groups are what you modify daily as you add services, change architectures, or troubleshoot connectivity issues. If you're troubleshooting connectivity, always check Security Groups first because that's where most issues are.

---

4. What happens when Auto Scaling launches a new instance?

When Auto Scaling decides to launch a new instance, it uses the launch template or launch configuration you defined. This template contains all the information needed - the AMI to use, instance type, security groups, IAM role, user data script, and which subnets to launch into.

Auto Scaling picks an availability zone, usually choosing the one with the fewest instances to maintain balance. It then launches an EC2 instance in a subnet in that AZ using the specifications from your template. If you specified user data in the template, that script runs when the instance boots up - this typically installs software, configures the application, and starts services.

While the instance is starting up, Auto Scaling marks it as "Pending" or "Initializing." During this time, the instance isn't receiving any traffic yet. Auto Scaling waits for the instance to pass its health checks before considering it healthy and ready.

Once the instance is running, the health check process begins. If you're using a load balancer, Auto Scaling waits for the load balancer's health check to pass. The load balancer sends requests to the instance's health check endpoint - maybe hitting /health or just checking if the web server responds on port 80. The instance needs to pass several consecutive health checks before being marked healthy.

After the instance passes health checks, Auto Scaling registers it with the load balancer's target group. Now the load balancer starts sending real user traffic to it. The traffic distribution gradually includes the new instance along with existing ones.

If the instance fails to launch or fails health checks, Auto Scaling automatically terminates it and tries again. It keeps attempting until it successfully launches a healthy instance or hits your maximum capacity limit. All of this happens automatically without any manual intervention - you just see your desired capacity increase and new instances appear.

---

5. How does Load Balancer decide where to send traffic?

The load balancer uses a routing algorithm combined with health check status to decide where to send each request. The most common algorithm is round-robin, where it cycles through healthy instances in order. If you have three instances, the first request goes to instance A, second to instance B, third to instance C, then back to instance A, and so on.

Before sending traffic anywhere, the load balancer checks which instances are healthy. It continuously sends health check requests to each instance - maybe hitting a /health endpoint every 30 seconds. If an instance fails a certain number of consecutive health checks, it's marked unhealthy and removed from rotation. The load balancer won't send any traffic to unhealthy instances until they pass health checks again.

Application Load Balancers have more sophisticated routing. You can set up rules based on the URL path. For example, requests to /api/* go to your API servers, requests to /images/* go to your image servers, and everything else goes to your web servers. You can also route based on hostnames, HTTP headers, or query parameters.

There's also least outstanding requests algorithm, where the load balancer sends traffic to the instance that currently has the fewest active connections. This is useful when requests take varying amounts of time to process - you don't want one instance getting stuck with all the slow requests while others sit idle.

For sticky sessions, the load balancer can use cookies to ensure a user's requests always go to the same instance. When a user first connects, the load balancer picks an instance and sets a cookie. Subsequent requests from that user include the cookie, and the load balancer routes them to the same instance. This is useful for applications that store session data locally on the instance.

The load balancer also considers availability zones. It tries to distribute traffic evenly across AZs to prevent overloading instances in one zone. If you have 2 instances in AZ-A and 4 instances in AZ-B, it doesn't just round-robin through all 6 - it balances across zones first, then across instances within each zone.

---

6. What is the difference between horizontal and vertical scaling?

Horizontal scaling means adding more instances of the same size. If you have one t3.medium instance and need more capacity, you add a second t3.medium, then a third, and so on. You're scaling by quantity - more servers doing the same work.

Vertical scaling means making your instance bigger. Instead of adding more t3.medium instances, you stop your one instance and change it to t3.large, then t3.xlarge, then t3.2xlarge. You're scaling by size - making the single server more powerful.

The big difference is in availability. With horizontal scaling, if one instance fails, the others keep running and your application stays up. With vertical scaling, you only have one instance, so if it fails, everything goes down. There's no redundancy.

Cost scaling is different too. Horizontal scaling is linear - 4 small instances cost about 4x what 1 small instance costs. Vertical scaling gets expensive fast - a t3.2xlarge might cost 8x more than a t3.medium but doesn't give you 8x the performance due to overhead and diminishing returns.

Limits are another factor. With horizontal scaling, you can keep adding instances almost indefinitely - AWS will let you run hundreds or thousands of instances. With vertical scaling, you eventually hit the biggest instance type available. If you're already on the largest instance and need more capacity, you're stuck.

Downtime is a practical concern. Vertical scaling requires stopping the instance, changing its type, and restarting it - that's several minutes of downtime. Horizontal scaling is seamless - you launch new instances without touching existing ones, so there's no downtime.

For most modern applications, horizontal scaling is preferred. It's more reliable, more cost-effective at scale, and works better with cloud-native architectures like microservices. You use vertical scaling when you have specific constraints, like a database that can't be easily distributed or legacy applications that can't run on multiple servers.

---

7. How would you connect two VPCs?

The most common way to connect two VPCs is VPC Peering. You create a peering connection between the VPCs, which establishes a private network connection between them. Traffic flows directly between the VPCs using AWS's private network, never touching the public internet.

To set up VPC peering, you first create the peering connection request from one VPC to the other. The owner of the second VPC accepts the request. Then you update the route tables in both VPCs to add routes pointing to each other. For example, if VPC-A is 10.0.0.0/16 and VPC-B is 10.1.0.0/16, you add a route in VPC-A's route table saying "for 10.1.0.0/16, send traffic to the peering connection." You do the reverse in VPC-B.

Security groups need updating too. By default, security groups only reference CIDR blocks within their own VPC. You need to add rules allowing traffic from the other VPC's CIDR range. Or better yet, you can reference security groups from the peered VPC if they're in the same region.

One important limitation - VPC peering is not transitive. If VPC-A peers with VPC-B, and VPC-B peers with VPC-C, VPC-A cannot talk to VPC-C through VPC-B. You'd need a direct peering connection between VPC-A and VPC-C. This can get complicated with many VPCs.

For connecting many VPCs, AWS Transit Gateway is better. It acts as a hub where you connect multiple VPCs, and they can all communicate through it. Instead of creating peering connections between every pair of VPCs (which grows exponentially), you just connect each VPC to the Transit Gateway once. It's simpler to manage and supports transitive routing.

Another option is VPN connections if the VPCs are in different AWS accounts or you need encryption. You set up VPN gateways in each VPC and establish VPN tunnels between them. This is more complex and has lower throughput than peering, but gives you encrypted connections.

Make sure CIDR blocks don't overlap. If both VPCs use 10.0.0.0/16, you can't peer them because routing would be ambiguous. This is why planning your IP addressing scheme upfront is important.

---

8. What is the difference between IAM Role and IAM User?

IAM Users are for people - actual humans who need to access AWS. When you create a user, you give them a username and password for console access, and optionally access keys for programmatic access. Users are long-term identities that exist until you delete them.

IAM Roles are for applications, services, or temporary access. Roles don't have permanent credentials. Instead, when something assumes a role, it gets temporary credentials that expire after a set time, usually a few hours. These credentials automatically rotate.

The key difference is in how credentials work. With a user, you have access keys that are permanent until you manually rotate them. These keys can be stolen, leaked in code, or compromised. With a role, credentials are temporary and automatically rotated by AWS, so even if they're stolen, they only work for a short time.

For EC2 instances, you always use roles, never users. You attach a role to the instance, and applications running on it automatically get temporary credentials from the instance metadata service. The application doesn't need any hardcoded keys - it just calls AWS APIs and the credentials are provided automatically. When the credentials expire, AWS automatically refreshes them.

Users are for your team members. Your developers get IAM users to access the AWS console or use the CLI from their laptops. You'd create a user for each person, put them in groups with appropriate permissions, and enable MFA for security.

Roles are also used for cross-account access. If you have multiple AWS accounts and need resources in Account A to access resources in Account B, you create a role in Account B that Account A can assume. This is much cleaner than creating users in every account.

Another common use is for AWS services. When Lambda needs to write to S3, you attach a role to the Lambda function with S3 permissions. When CodePipeline needs to deploy to EC2, it assumes a role with the necessary permissions. Services can't use IAM users - they must use roles.

In practice, you create users for people and roles for everything else. This follows the principle of least privilege and reduces the risk of credential exposure.

---

9. How does DNS resolution work?

DNS resolution is the process of converting a human-readable domain name like www.example.com into an IP address that computers can use. When you type a URL in your browser, this process happens behind the scenes.
First, your browser checks its cache to see if it already knows the IP address for that domain. If you visited the site recently, the IP might still be cached. If not cached, the browser asks your operating system, which has its own DNS cache.

If the OS doesn't have it cached, it sends a query to your configured DNS resolver, usually provided by your ISP or a public DNS service like Google (8.8.8.8) or Cloudflare (1.1.1.1). This resolver is called a recursive resolver because it does the work of finding the answer for you.

The recursive resolver starts by asking the root DNS servers. There are 13 root server systems worldwide that know about all top-level domains (.com, .org, .net, etc.). The root server responds with "I don't know the IP for www.example.com, but I know the DNS servers for .com domains."
The resolver then queries the .com TLD (Top Level Domain) servers. These servers respond with "I don't know the IP, but I know the authoritative name servers for example.com." If you're using Route 53, it would point to AWS's name servers.

Finally, the resolver queries the authoritative name server for example.com. This server has the actual DNS records and responds with the IP address, maybe 54.123.45.67. It also includes a TTL (Time To Live) that says how long this answer can be cached.

The resolver returns the IP to your OS, which returns it to your browser. Your browser then caches it, and the OS caches it, so future requests are faster. Then your browser can make an HTTP request to that IP address to load the

10. What is NAT Gateway and why is it used?

A NAT Gateway is a managed service that allows instances in private subnets to access the internet for outbound connections while preventing inbound connections from the internet. NAT stands for Network Address Translation, and it translates private IP addresses to public ones.

Here's why you need it - instances in private subnets don't have public IP addresses and their route tables don't point to an Internet Gateway. So they can't reach the internet directly. But they often need internet access for things like downloading software updates, calling external APIs, or accessing AWS services that use public endpoints.

The NAT Gateway sits in a public subnet and has both a private IP and an Elastic IP (public IP). When an instance in a private subnet wants to reach the internet, its traffic is routed to the NAT Gateway based on the private subnet's route table. The NAT Gateway receives the request, replaces the source IP with its own public IP, and forwards it to the Internet Gateway. When the response comes back, the NAT Gateway translates it back to the original private IP and sends it to the instance.

The key security feature is that NAT Gateway only allows outbound connections. If someone on the internet tries to connect to your NAT Gateway's public IP, they can't reach your private instances. The NAT Gateway blocks all inbound connection attempts. It only allows response traffic for connections that were initiated from inside your VPC.

In production, you should deploy NAT Gateways in multiple availability zones for high availability. If you only have one NAT Gateway and that AZ goes down, instances in other AZs lose internet access. So you'd typically put a NAT Gateway in each AZ and configure route tables so each private subnet uses the NAT Gateway in its own AZ.

NAT Gateways are fully managed by AWS - they're highly available within an AZ, automatically scale up to 45 Gbps, and you don't need to patch or maintain them. You just pay hourly charges plus data processing charges. The alternative is a NAT Instance, which is an EC2 instance you manage yourself, but that's rarely used anymore because NAT Gateways are more reliable and easier.

---

11. What are S3 storage classes and when would you use them?

S3 has different storage classes optimized for different access patterns and cost requirements. Choosing the right class can save you significant money.

S3 Standard is for frequently accessed data. It's the default and most expensive storage class, but gives you immediate access with high performance. You'd use this for active data like website assets, content that users access regularly, or data you're actively processing. It's designed for 99.99% availability and stores data across at least 3 availability zones.
S3 Intelligent-Tiering automatically moves data between frequent and infrequent access tiers based on usage patterns. If you haven't accessed an object in 30 days, it moves to a cheaper tier. If you access it again, it moves back. This is great when you don't know or can't

predict access patterns. There's a small monitoring fee per object, so it works best for larger objects.
S3 Standard-IA (Infrequent Access) is for data you don't access often but need immediately when you do. It's about 50% cheaper than Standard for storage, but you pay a retrieval fee per GB. You'd use this for backups, disaster recovery files, or older data that's rarely needed but must be available quickly. There's a minimum storage duration of 30 days and minimum object size of 128KB.
S3 One Zone-IA is similar to Standard-IA but stores data in only one availability zone instead of three, making it about 20% cheaper. Use this for data you can easily recreate if lost, like thumbnails generated from original images or secondary backup copies. If that AZ is destroyed, your data is gone.
S3 Glacier Instant Retrieval is for archive data that you rarely access but need within milliseconds when you do. It's cheaper than Standard-IA but has higher retrieval costs. Good for medical images or media archives that must be immediately available but are rarely accessed. Minimum storage duration is 90 days.
S3 Glacier Flexible Retrieval (formerly Glacier) is for archives where you can wait minutes to hours for retrieval. It's very cheap for storage. You have retrieval options - expedited (1-5 minutes, expensive), standard (3-5 hours, moderate cost), or bulk (5-12 hours, cheapest). Use this for compliance archives, old backups, or historical data. Minimum storage duration is 90 days.
S3 Glacier Deep Archive is the cheapest option, about $1 per TB per month. Retrieval takes 12-48 hours. This is for data you almost never access but must retain for compliance - think 7-10 year retention requirements for financial or medical records. Minimum storage duration is 180 days.
You can set up lifecycle policies to automatically transition objects between classes. For example, move logs to Standard-IA after 30 days, to Glacier after 90 days, and delete after 7 years. This automation saves money without manual intervention.

---

12. How do you secure an EC2 instance?

Securing an EC2 instance involves multiple layers starting from network security down to the operating system and application level.

At the network layer, use Security Groups properly. Only open the ports you absolutely need - if it's a web server, allow 80 and 443 from anywhere, but restrict SSH (port 22) to only your office IP or use AWS Systems Manager Session Manager instead so you don't need SSH open at all. Put instances in private subnets whenever possible so they're not directly accessible from the internet.

Use IAM roles instead of access keys. Attach a role to your instance with only the permissions it needs. If it only needs to read from a specific S3 bucket, give it only that permission, not full S3 access. Never hardcode AWS credentials in your application or configuration files.

Keep your operating system and software updated. Enable automatic security updates for critical patches. Use AWS Systems Manager Patch Manager to automate patching across multiple instances. Outdated software is one of the most common ways instances get compromised.

Disable root login and password authentication for SSH. Use SSH key pairs instead, and protect those private keys. Consider using AWS Systems Manager Session Manager which lets you access instances without SSH keys at all and provides full audit logging of every session.

Enable detailed monitoring and logging. Send logs to CloudWatch Logs so you can detect suspicious activity. Enable VPC Flow Logs to see all network traffic. Set up CloudWatch alarms for unusual patterns like high CPU usage, excessive network traffic, or failed login attempts.

Encrypt data at rest and in transit. Use encrypted EBS volumes for your instance storage. Enable HTTPS for web traffic using certificates from AWS Certificate Manager. For sensitive data, consider using AWS KMS for encryption key management.

Use separate instances for different functions. Don't run your web server and database on the same instance. This limits the blast radius if one component is compromised. Follow the principle of least privilege everywhere - each component should only have access to what it needs.

Implement a bastion host or jump box architecture. Instead of allowing SSH to all instances, only allow it to a hardened bastion host in a public subnet, then SSH from there to instances in private subnets. Better yet, use Systems Manager Session Manager and eliminate SSH entirely.

Regular security scanning is important. Use AWS Inspector to automatically scan for vulnerabilities and compliance issues. Run third-party security tools to check for misconfigurations. Perform penetration testing periodically.

Have a backup and disaster recovery plan. Take regular snapshots of your EBS volumes. Store backups in S3 with versioning enabled. Test your restore process to make sure backups actually work when you need them.

.

13. What is Infrastructure as Code?

Infrastructure as Code (IaC) is managing infrastructure through code files instead of manual console clicks. You write code defining your VPC, subnets, EC2 instances, etc., and tools automatically create them.

Key Benefits:

Consistency: Deploy the same code multiple times, get identical infrastructure every time Version Control: Store in Git, track changes, roll back if needed Speed: Deploy environments in minutes, not days of manual setup Documentation: Code itself documents your infrastructure

Common Tools:

CloudFormation (AWS-specific, YAML/JSON) Terraform (multi-cloud, HCL language) AWS CDK (use Python, TypeScript, etc.)

Example Use: Instead of manually clicking through AWS console to create a VPC, you write it in Terraform once and can deploy identical environments in dev, staging, and production. If disaster strikes, redeploy to another region in minutes.

---

14. What is Docker and why is it used?

Docker packages your application and all dependencies into containers - lightweight, portable units that run identically everywhere. It solves the "works on my machine" problem.

Key Differences from VMs:

Containers share host OS, VMs include full OS Containers are MB in size and start in seconds VMs are GB in size and take minutes to start Run dozens of containers vs few VMs per server

How It Works:

Write a Dockerfile defining your environment → Build into an image → Run image to create containers. Store images in registries (Docker Hub, ECR) and deploy anywhere.

Why Use It:

Microservices: Each service runs in its own container with exact dependencies needed Development: New developers run docker-compose up and get complete environment instantly Deployment: Same container image works in dev, staging, production Resource Efficiency: Better server utilization than VMs

In AWS, use ECS or EKS to run containers at scale.

---

15. What is Kubernetes and why do we need it?

Kubernetes (K8s) orchestrates containers at scale. While Docker runs containers, Kubernetes manages thousands of them across multiple servers automatically.

What It Does:

Auto-healing: Crashed containers automatically restart Auto-scaling: Scale from 10 to 100 containers based on CPU/traffic Load Balancing: Built-in traffic distribution Rolling Updates: Deploy new versions with zero downtime Self-healing: Failed nodes trigger automatic container rescheduling

Key Features:

Service discovery: Containers find each other by name, not IP Secrets management: Store credentials separately from code Configuration: Same image works across environments with different configs

When You Need It:

Small apps with few containers? Use Docker and ECS. Complex apps with many microservices and thousands of containers? Kubernetes provides the orchestration layer to manage it all.

In AWS, use EKS (managed Kubernetes) where AWS handles the control plane.

---

16. Traffic Flow: Internet to Private EC2 Behind Load Balancer

Step-by-step flow:

1. DNS Resolution: User types URL → Route 53 returns ALB's public IP
2. Internet Gateway: Request enters VPC through IGW
3. Route Table: Directs traffic to public subnet
4. NACL Check: Public subnet NACL allows HTTPS (port 443)
5. Load Balancer: ALB in public subnet receives request, terminates SSL
6. Target Selection: ALB picks healthy instance via round-robin
7. Route to Private Subnet: Traffic routed using VPC local route
8. Private NACL: Checks inbound rules (stateless)
9. Security Group: Instance SG allows traffic from ALB's security group
10. Application: EC2 instance processes request
11. Response Back: Security Group auto-allows response (stateful)
12. NACL Outbound: Needs explicit outbound rule (stateless)
13. ALB Encrypts: Re-encrypts response and sends to user

Key Security: EC2 has no public IP, lives in private subnet, only accessible through ALB. No direct internet exposure.

17. How to Allow Private EC2 to Download Updates Securely?

NAT Gateway (Most Common):

Place NAT Gateway in public subnet with Elastic IP Update private subnet route table: 0.0.0.0/0 → NAT Gateway Instance traffic goes out through NAT, but no inbound connections allowed Deploy NAT Gateway in each AZ for high availability

VPC Endpoints (For AWS Services):

Gateway Endpoints (S3, DynamoDB): Free, traffic stays in AWS network Interface Endpoints (Systems Manager, Secrets Manager): Traffic never touches internet More secure and often cheaper than NAT Gateway

Hybrid Approach (Best Practice):

VPC Endpoints for AWS services NAT Gateway for internet access (OS updates, external APIs) Systems Manager Session Manager for instance access (no SSH needed)

Security: Use Security Groups to limit outbound connections to only necessary destinations.

18. How Route Tables Decide Where Traffic Goes

Route tables use longest prefix matching - most specific route wins.

Example Route Table:

10.0.0.0/16 → local 0.0.0.0/0 → igw-123456

Traffic to 10.0.5.20 → matches both, but /16 more specific than /0 → stays local in VPC Traffic to 8.8.8.8 → only matches 0.0.0.0/0 → goes to Internet Gateway

Common Configurations:

Public Subnet:

10.0.0.0/16 → local 0.0.0.0/0 → igw (Internet Gateway)

Private Subnet:

10.0.0.0/16 → local 0.0.0.0/0 → nat (NAT Gateway)

With VPN:

10.0.0.0/16 → local 192.168.0.0/16 → vgw (VPN Gateway) 0.0.0.0/0 → nat

Priority: /32 > /24 > /16 > /0. Local routes always win, then peering, VPN, then IGW/NAT.

Best Practice: Keep main route table private (no IGW route) so new subnets are secure by default.

---

19. When to Modify NACL Instead of Security Group?

Use NACLs for:

1.Blocking Specific IPs:

Security Groups only allow rules. To block malicious IPs during an attack:

Rule 50: DENY 203.0.113.0/24 Rule 100: ALLOW 0.0.0.0/0

2.Compliance Requirements:

Enforce that certain subnets can never communicate, even if Security Groups are misconfigured. NACL provides subnet-level enforcement.

3.Subnet-Wide Policies:

Apply blanket rules to all resources in a subnet without touching individual Security Groups.

Key Differences:

Security Groups: Stateful, instance-level, allow-only, primary defense NACLs: Stateless, subnet-level, allow/deny, secondary defense

Real-World Use: Set up NACLs once during VPC setup, rarely modify. Use Security Groups for day-to-day security management.

---

20. Connecting On-Prem Data Center to Cloud Securely

Site-to-Site VPN (Quick Start):

Encrypted tunnel over public internet Setup: Virtual Private Gateway (AWS) + Customer Gateway (on-prem router) Pros: Quick (hours), cheap ($0.05/hour), encrypted by default Cons: Limited bandwidth, variable latency Update route tables on both sides for traffic flow

AWS Direct Connect (Production):

Dedicated private fiber connection 1-10 Gbps, can go up to 100 Gbps Pros: Consistent low latency, higher bandwidth, lower data transfer costs Cons: Expensive, takes weeks to provision, complex setup Not encrypted by default (can run VPN over it)

Hybrid Approach (Recommended):

Direct Connect as primary VPN as backup Automatic failover if Direct Connect fails

Transit Gateway (Multiple VPCs):

Instead of connecting each VPC individually, connect all VPCs to Transit Gateway, then connect data center once to Transit Gateway. Simplifies management dramatically.

Security Considerations:

Ensure CIDR blocks don't overlap Use Security Groups/NACLs to control access Enable VPC Flow Logs for monitoring Consider AWS Network Firewall for traffic inspection

---

Topic: Scaling & Availability

21. How Does Auto Scaling Detect Unhealthy Instances?

EC2 Status Checks:

System status: Monitors physical host (power, network, hardware) Instance status: Monitors your instance (OS, networking, memory, filesystem) Checked every minute, unhealthy instances terminated after grace period (default 300s) Limitation: Only catches infrastructure issues, not application failures

ELB Health Checks (Recommended):

Load balancer sends requests to /health endpoint every 30 seconds Application must respond with 200 OK Thresholds: 2 consecutive failures = unhealthy, 10 consecutive successes = healthy Catches application-level failures EC2 checks miss

Health Check Grace Period:

Time for new instances to boot and start before health checks begin Set based on your application's startup time (e.g., 180s for 2-minute startup) Too short = instances terminated before ready; too long = unhealthy instances linger

Custom Health Checks:

Your /health endpoint should verify: Database connectivity Required services running Disk space available Return 503/500 if any critical component fails

What Happens:

Unhealthy instance → Auto Scaling terminates it → Launches replacement → New instance goes through health checks → Added to load balancer when healthy

---

22. What Happens if One Availability Zone Fails?

Well-Designed Multi-AZ Architecture:

Load Balancer:

Detects instances in failed AZ are unhealthy within seconds Stops sending traffic to failed AZ Routes all traffic to healthy AZs Users experience minimal disruption

Auto Scaling:

Detects unhealthy instances in failed AZ Can't launch in failed AZ, so launches replacements in healthy AZs Desired capacity maintained, redistributed across working AZs

RDS Database:

If primary in failed AZ, automatic failover to standby replica in another AZ Takes 60-120 seconds DNS automatically points to new primary Brief write outage during failover

NAT Gateway:

If you have NAT in each AZ, instances in healthy AZs continue internet access Instances in failed AZ lose internet access (but they're unhealthy anyway) Best Practice: Deploy NAT Gateway in each AZ

S3:

Automatically replicates across AZs Continues serving from other AZs seamlessly No action needed

User Impact:

Brief slowdown when AZ first fails Some errors during database failover (1-2 minutes) Back to normal within minutes as Auto Scaling adds capacity

Poorly Designed Single-AZ:

Complete outage until AWS restores the AZ. Never acceptable for production.

---

23. How to Ensure Zero Single Point of Failure?

Compute Layer:

Minimum 2 instances per service, preferably 3+ Spread across multiple AZs (e.g., 2 in us-east-1a, 2 in us-east-1b) Use Auto Scaling to automatically replace failed instances Avoid spot instances for critical components

Load Balancer:

ALB/NLB automatically highly available across AZs Enable multiple AZs when creating Configure aggressive health checks for fast failure detection

Database:

RDS: Enable Multi-AZ for automatic failover Aurora: Replicates across 3 AZs by default, faster failover Self-managed: Master-slave replication across AZs Regular automated backups

Storage:

EBS: Lives in single AZ, use snapshots for backup EFS: Automatically replicates across AZs S3: Automatically replicates across multiple AZs

Network:

NAT Gateway in each AZ Multiple VPN connections or Direct Connect + VPN backup Route 53 health checks with failover routing

Application Design:

Stateless applications (store session data in ElastiCache/DynamoDB) Circuit breakers for graceful degradation Retry logic with exponential backoff Queue-based architectures for decoupling

Monitoring:

CloudWatch alarms for each critical component Alert on patterns indicating AZ issues Regular chaos engineering tests

Cost vs. Availability:

Multi-AZ costs more but downtime costs more than infrastructure. Balance based on SLA requirements.

---

24. How Does a Load Balancer Perform Health Checks?

Configuration:

Target: Path to check (e.g., /health, /status) Interval: How often to check (default 30 seconds) Timeout: How long to wait for response (default 5 seconds) Healthy Threshold: Consecutive successes needed (default 10) Unhealthy Threshold: Consecutive failures needed (default 2)

Process:

1.Load balancer sends HTTP GET to /health every 30 seconds

2.Instance must respond with 200 OK within 5 seconds

3.After 2 consecutive failures → marked unhealthy, removed from rotation

4.After 10 consecutive successes → marked healthy, added to rotation

Why Different Thresholds:

Quick to mark unhealthy (2 failures) = fast problem detection Slow to mark healthy (10 successes) = prevents flapping from intermittent issues

Health Check Endpoint Best Practices:

@app.route('/health') def health_check(): # Check database if not db.ping(): return "DB down", 503

```
# Check Redis
if not redis.ping():
    return "Redis down", 503

# Check disk space
if disk_usage > 90:
    return "Disk full", 503

return "OK", 200
```

Advanced Features:

Custom ports: Health check different port than traffic Custom headers: Add authentication headers Path-based: Different health checks for different target groups gRPC health checks: For gRPC applications

Monitoring:

CloudWatch metrics show healthy/unhealthy target counts Set alarms when healthy targets drop below threshold Review health check logs to diagnose failures

---

25. When Would Vertical Scaling Be a Bad Idea?

Bad Situations for Vertical Scaling:

1.High Availability Requirements:

Single larger instance = single point of failure Downtime required to resize If instance fails, entire application down Better: Horizontal scaling with multiple instances

2.Already at Maximum Instance Size:

AWS has limits (e.g., largest instance might be 448 vCPUs) Can't scale beyond that Better: Horizontal scaling has no practical limit

3.Cost Inefficiency:

Large instances don't scale linearly in performance t3.2xlarge costs 8x more than t3.medium but doesn't give 8x capacity Wasted resources during low traffic periods Better: Horizontal scaling with right-sized instances

4.Uneven Resource Usage:

Application might be CPU-bound but not memory-bound Vertical scaling increases both unnecessarily Better: Horizontal scaling or different instance types

5.Microservices Architecture:

Different services have different resource needs Vertical scaling treats everything the same Better: Horizontal scaling per service

6.Elastic Workloads:

Traffic varies significantly (10x difference peak vs. off-peak) Vertical scaling means paying for peak capacity 24/7 Better: Horizontal auto-scaling adjusts to demand

7.Deployment Risk:

Vertical scaling requires stopping instance All eggs in one basket during deployment Better: Horizontal scaling with rolling deployments

When Vertical Scaling IS Good:

Single-threaded applications that can't distribute Databases with complex transactions requiring single instance Legacy applications that can't run distributed Quick temporary fix before refactoring for horizontal scaling

Best Practice: Start with appropriately sized instances, then scale horizontally. Reserve vertical scaling for specific bottlenecks.

Topic: Storage & Data

26. How do you decide between EBS and EFS for an application?

EBS is block storage that is attached to a single EC2 instance and is used like a physical disk by the operating system, providing very low latency and high performance. It is commonly used for databases, boot volumes, and applications that require fast and consistent disk access. Since EBS volumes are tied to a single instance, they are best suited for workloads that do not require shared access.

EFS is a fully managed network file system that can be mounted by multiple EC2 instances at the same time and automatically scales as data grows. It is highly available across multiple Availability Zones and is typically used when applications need shared storage across instances.

Use EBS when:

Storage is needed by one EC2 instance
Low latency and high performance are critical
Use EFS when:

Multiple EC2 instances need shared file access
Applications require scalable, shared storage

27. How would you design a backup strategy for a production database?

A production database backup strategy should be designed to protect against accidental deletion, data corruption, and large-scale failures such as an Availability Zone or region outage. Backups should always be automated, securely stored, and regularly tested to ensure that data can actually be restored when needed.

For managed databases like Amazon RDS, automated backups should be enabled to allow point-in-time recovery, with a retention period based on business requirements. In addition to automated backups, manual snapshots should be taken before major changes such as schema updates or application deployments, and these snapshots should be copied to another region for disaster recovery. Enabling Multi-AZ deployments provides high availability, although it should not be treated as a replacement for backups.

For databases running on EC2 instances, backups are typically implemented using EBS snapshots. These snapshots should be scheduled using AWS Backup or automation tools and are incremental, reducing storage costs. To further improve resilience, snapshots should be copied to another region and optionally stored in a separate AWS account. All backups should be encrypted using AWS KMS, access should be restricted using IAM policies, and restore procedures should be tested regularly to validate recovery time and data integrity.

---

28. How does object storage differ architecturally from block storage?

Block storage is designed to function like a traditional hard drive and is attached to a compute instance such as EC2. Data is stored in fixed-size blocks, and the operating system is responsible for managing the file system, including formatting and mounting the volume. This architecture provides very low latency and high performance, making block storage ideal for databases, operating system disks, and applications that require fast, consistent disk access.

Object storage is architected in a fundamentally different way. Instead of blocks, data is stored as objects, where each object contains the data itself along with metadata and a unique identifier. Object storage is accessed through APIs such as HTTP rather than a file system and does not require a server or operating system to manage storage. This design allows object storage to scale almost infinitely while offering extremely high durability and availability.

Because of these architectural differences, block storage is best suited for performance-critical, server-attached workloads, while object storage is ideal for backups, media files, logs, static content, and large-scale data storage where scalability and durability are more important than low latency.

---

29. When would you move data to cheaper storage tiers?

Data should be moved to cheaper storage tiers when it is no longer accessed frequently but still needs to be retained for operational, legal, or compliance reasons. This commonly includes older application logs, historical data, archived backups, audit records, and inactive user data that must be kept for long periods but is rarely retrieved.

In AWS, this is typically handled using lifecycle policies that automatically transition data from standard storage tiers to infrequent access and archival tiers such as S3 Standard-IA, Glacier, or Glacier Deep Archive. These transitions significantly reduce storage costs while still allowing data to be retrieved when necessary, although with higher access latency and retrieval costs.

Data should not be moved to cheaper tiers if it is accessed frequently, is latency-sensitive, or consists of small objects where retrieval costs may outweigh storage savings. The key factor in deciding is understanding access patterns over time and balancing cost savings against performance and availability requirements.

---

30. How would you prevent accidental deletion of important data?

Preventing accidental deletion of important data requires a defense-in-depth approach that combines access controls, recovery mechanisms, and monitoring. Enabling versioning ensures that deleted or overwritten data can be recovered, protecting against both human error and application mistakes.

Strict IAM policies should be applied using the principle of least privilege so that delete permissions are granted only to authorized roles. For highly critical or regulated data, additional protection mechanisms such as MFA Delete or S3 Object Lock can be used to prevent deletion for a defined retention period, even by administrators.

In addition to these controls, regular backups should be maintained and stored in a separate AWS account or region to protect against account-level failures. Monitoring and logging using CloudTrail should be enabled to track delete operations, and alerts should be configured to notify teams of suspicious or unexpected delete activity. This layered approach ensures that even if a deletion occurs, the data can still be recovered.

Topic: Security & IAM:

31. How would you design least-privilege access for a team?

Designing least-privilege access means ensuring that users and services have only the permissions they need to perform their tasks and nothing more. The first step is to understand roles and responsibilities within the team, such as developers, testers, DevOps engineers, and administrators, and identify what actions each role actually needs to perform.

Access should be granted using IAM roles and policies rather than individual permissions. Instead of assigning broad permissions like full access, policies should be scoped to specific services, actions, and resources. For example, developers may need read access to production logs but no permission to modify infrastructure, while deployment roles may need write access only during deployments.

Permissions should be reviewed regularly to remove unused or excessive access, and temporary elevated access should be granted only when required. Using IAM roles, permission boundaries, and approval workflows helps ensure access remains controlled and auditable.

Key principles include:

Grant minimum required permissions
Use roles instead of individual users where possible
Avoid wildcard permissions like *
Regularly review and audit access

---

32. How do temporary credentials improve security?

Temporary credentials improve security by providing short-lived access instead of long-term static credentials. These credentials are issued for a limited duration and automatically expire, which significantly reduces the risk if they are exposed or compromised.

In AWS, temporary credentials are commonly provided through IAM roles and AWS STS. When a user, application, or service assumes a role, it receives temporary access keys that are valid only for a specific time window. Once expired, they can no longer be used, eliminating the need for manual key rotation.

Temporary credentials are especially useful for applications running on EC2, ECS, or Lambda, because credentials are automatically managed and never stored in code or

configuration files. This reduces the attack surface and enforces better security practices by design.

Benefits of temporary credentials include:

Automatic expiration
No hard-coded secrets
Reduced impact of credential leaks
Easier credential rotation

---

33. How would you rotate secrets in a running application?

Rotating secrets in a running application should be done in a way that avoids downtime and does not require redeploying code. This is typically achieved by using a centralized secrets management service such as AWS Secrets Manager or Parameter Store.

Secrets are stored securely and accessed dynamically by the application at runtime rather than being hard-coded. Rotation can be automated so that new credentials are generated and stored while the old ones remain valid for a short overlap period. During this time, the application can seamlessly switch to the new secret.

Applications should be designed to fetch secrets at startup or periodically refresh them from the secrets service. IAM roles are used to control which applications can access which secrets, ensuring secure access without embedding credentials.

Key practices include:

Store secrets centrally, not in code
Enable automated rotation
Allow overlap between old and new secrets
Ensure applications can reload secrets without restart

---

34. What security layers exist in a cloud architecture?

Cloud security is based on a layered, defense-in-depth approach where multiple controls work together to protect systems and data. Each layer is designed to reduce risk even if another layer fails.

At the network layer, controls such as VPC isolation, security groups, network ACLs, and firewalls restrict traffic flow. At the identity layer, IAM policies, roles, MFA, and least-privilege access control who can access resources. At the compute and application

layer, patching, secure configurations, and application-level authentication protect workloads.

Data security is handled through encryption at rest and in transit, key management services, and backup strategies. Monitoring and logging form another layer by detecting threats and enabling rapid response through services like CloudTrail, CloudWatch, and security monitoring tools.

Key security layers include:

Network security
Identity and access management
Compute and application security
Data protection and encryption
Monitoring and logging

---

35. How would you isolate environments (dev, test, prod) securely?

Environment isolation is critical to prevent accidental changes, data leaks, or security breaches from affecting production systems. The most secure approach is to separate environments using different AWS accounts, which provides strong isolation at the account boundary.

Each environment should have its own VPC, IAM roles, and resources, ensuring that access to production is strictly limited. Networking between environments should be controlled and minimized, and production data should never be accessible from development or test environments.

IAM policies should enforce strict access controls so that developers have limited access to production, while automation and CI/CD pipelines handle deployments using controlled roles. Separate monitoring, logging, and encryption keys should also be used per environment.

Best practices include:

Use separate AWS accounts per environment
Restrict production access with IAM and MFA
Avoid sharing data between environments
Deploy using automated pipelines, not manual access

Topic: DevOps & Containers (Optional but good to have basic understanding)

36. Why is containerization better than deploying directly on VMs?

Containerization is better than deploying directly on virtual machines because it packages the application and all its dependencies together, ensuring the application runs consistently across different environments. When deploying directly on VMs, applications often depend on the underlying operating system configuration, which can lead to issues such as "it works on my machine but not in production."

Containers are lightweight compared to VMs because they share the host operating system kernel instead of running a full OS for each application. This allows faster startup times, better resource utilization, and higher application density on the same infrastructure. Containers also make deployments and rollbacks faster because applications are delivered as immutable images rather than manually configured servers.

Overall, containerization improves portability, consistency, scalability, and operational efficiency compared to traditional VM-based deployments.

---

37. What problem does Kubernetes solve that Docker alone doesn't?

Docker solves the problem of packaging and running containers, but it does not manage containers at scale. Kubernetes solves the orchestration problem by managing how containers are deployed, scaled, healed, and networked across multiple machines.

In a production environment with many containers, failures are inevitable—containers crash, nodes go down, traffic spikes, and updates are required without downtime. Kubernetes automatically handles these scenarios by restarting failed containers, rescheduling workloads when nodes fail, scaling applications based on demand, and performing rolling updates safely.

In short, Docker helps you build and run containers, while Kubernetes ensures those containers run reliably and efficiently in production at scale.

---

38. How would you implement rolling deployment?

Rolling deployment is implemented by gradually replacing old application versions with new ones instead of updating everything at once. This ensures that the application remains available during deployments and reduces the risk of outages.

In containerized environments such as Kubernetes, rolling deployments are achieved by configuring a deployment strategy that specifies how many instances can be updated at a time. New pods are started with the updated version while old pods are terminated

gradually, ensuring a minimum number of healthy instances are always running. Health checks are used to confirm that new instances are ready before traffic is sent to them.

If issues are detected during the rollout, the deployment can be paused or rolled back to the previous version, minimizing user impact and deployment risk.

---

39. How does CI/CD reduce deployment risk?

CI/CD reduces deployment risk by automating and standardizing the build, test, and deployment process. Every change goes through the same pipeline, which reduces human error and ensures consistency across environments.

Continuous Integration ensures that code changes are automatically tested early, catching bugs before they reach production. Continuous Deployment or Delivery ensures that releases are small, frequent, and predictable, making failures easier to detect and recover from. Automated rollbacks, approval gates, and deployment strategies like blue-green or rolling deployments further reduce risk.

By removing manual steps and enforcing repeatable processes, CI/CD improves reliability, speeds up delivery, and makes deployments safer.

---

40. How would you design logging and monitoring for microservices?

Logging and monitoring for microservices should be centralized, scalable, and designed to provide visibility across distributed systems. Each microservice should produce structured logs that include contextual information such as request IDs, service names, and timestamps, allowing logs from multiple services to be correlated.

Logs should be collected centrally using tools like CloudWatch, ELK, or managed logging services so teams can search and analyze them from one place. Monitoring should include metrics such as latency, error rates, resource usage, and request throughput, along with health checks for each service.

Alerting should be based on meaningful thresholds and service-level indicators rather than individual host failures. Distributed tracing can also be added to track requests as they flow across services, helping quickly identify performance bottlenecks and failures in complex systems.

It means a lot! 💕